

3D Gaussian Splatting: Foundations, Anti-Aliasing, and Scene Editing

Dhwani Auvnish Chande
University of Illinois Chicago
dchan35@uic.edu

Trong Gia Hung Nguyen
University of Illinois Chicago
tnguy321@uic.edu

Nishita Konduru
University of Illinois Chicago
nkond5@uic.edu

Abstract

This report examines three interconnected advances in neural 3D scene representation and rendering: 3D Gaussian Splatting (3DGS) [3], Mip-Splatting [9], and Gaussian Grouping [8]. All three methods represent 3D scenes as collections of 3D Gaussian primitives, but each addresses a distinct limitation of the baseline approach. 3DGS establishes an explicit, differentiable rasterization pipeline enabling real-time novel view synthesis. Mip-Splatting resolves the aliasing artifacts produced by 3DGS under multi-scale rendering through principled signal-processing filters. Gaussian Grouping augments the Gaussian representation with identity-aware instance segmentation, enabling semantic scene understanding and downstream editing tasks. We implemented all three methods on the MipNeRF360 benchmark dataset [1] and on a self-captured desktop scene, evaluated them via PSNR, SSIM, and LPIPS metrics, and analyzed their respective strengths, limitations, and failure cases.

1. Introduction

1.1. Task

Novel view synthesis (NVS) is the task of generating photorealistic images of a 3D scene from camera viewpoints absent from the original capture. The input is a set of posed 2D photographs (RGB images paired with known or estimated camera intrinsics and extrinsics, typically recovered by structure-from-motion [6]). The system learns a 3D scene representation and synthesizes new views at arbitrary camera poses as output: photorealistic rendered images that match what a real camera would capture from those positions.

1.2. Applications

NVS has broad practical impact. Virtual and augmented reality systems benefit from free-viewpoint navigation of photorealistic environments. Autonomous vehicles and robots require accurate 3D scene understanding for naviga-

tion and manipulation. Telepresence platforms can reconstruct remote environments for immersive communication. In e-commerce and entertainment, interactive 3D product visualization and virtual set production become feasible without expensive manual 3D modeling. The scene-editing capabilities introduced by Gaussian Grouping further extend these applications to digital twins, post-production, and interactive simulation.

1.3. Challenges

NVS is non-trivial for several reasons. Recovering complete 3D structure from finite 2D projections is inherently under-constrained, requiring the model to infer unobserved geometry and appearance. Rendering must simultaneously achieve high perceptual quality and real-time speed, goals often in tension. Rendering the same scene at different resolutions or from farther distances than seen during training can introduce aliasing artifacts, degrading image quality. Finally, purely geometric representations lack semantic structure, limiting the ability to interact with or edit the reconstructed scene.

1.4. Why These Three Papers

We selected this set of papers because they form a natural progression addressing the core limitations of the 3DGS paradigm. **3DGS** [3] establishes the foundational explicit representation and differentiable rasterizer that made real-time NVS practical. **Mip-Splatting** [9] is a principled improvement that eliminates the scale-dependent aliasing produced by vanilla 3DGS, drawing on the frequency-domain theory of Mip-NeRF 360 [1]. **Gaussian Grouping** [8] extends the representation with identity-aware segmentation, enabling semantic understanding and editing. Together, the three papers cover the core rendering pipeline, its signal-processing correctness, and its extensibility, providing a comprehensive view of the current state and trajectory of Gaussian splatting research.

2. Approach 1: 3D Gaussian Splatting

2.1. Motivation

Prior to 3DGS, the dominant NVS paradigm was Neural Radiance Fields (NeRF) [5], which represents a scene as an implicit continuous function mapping 3D positions and viewing directions to color and density, approximated by a multi-layer perceptron. While NeRF achieves impressive image quality, inference requires querying the network at hundreds of sample points per ray, making real-time rendering on standard hardware infeasible. Subsequent methods improved speed via voxel grids or hash encodings but still fell short of interactive frame rates. The motivation for 3DGS was therefore to develop a representation that is both *explicit*, enabling fast, GPU-rasterizable rendering, and *flexible* enough to represent complex geometry and view-dependent appearance at competitive quality.

2.2. Novelty

The key innovations of 3DGS are threefold. First, scenes are represented as collections of explicit 3D Gaussian primitives, each defined by a mean position, a full 3D covariance matrix, an opacity value, and view-dependent color encoded via spherical harmonics. Second, rendering uses differentiable tile-based rasterization: Gaussians are projected to 2D, sorted by depth within screen-space tiles, and alpha-composited front-to-back, which is orders of magnitude faster than volumetric ray marching. Third, an adaptive densification strategy dynamically splits or clones Gaussians in under-reconstructed regions and prunes those with near-zero opacity, allowing the representation to grow or shrink to match scene complexity.

2.3. Approach

The pipeline initializes Gaussian positions from a sparse COLMAP point cloud. Each Gaussian is parameterized by a 3D mean μ , a covariance matrix $\Sigma = \mathbf{R}\mathbf{S}\mathbf{S}^\top\mathbf{R}^\top$ (where $\mathbf{S}$ is a diagonal scale matrix and $\mathbf{R}$ a rotation from a learned quaternion), an opacity $\sigma \in (0, 1)$, and spherical harmonic coefficients for view-dependent color. During rasterization, the 3D covariance is projected to a 2D covariance $\Sigma' = \mathbf{J}\mathbf{W}\Sigma\mathbf{W}^\top\mathbf{J}^\top$, where $\mathbf{J}$ is the Jacobian of the projective transform and $\mathbf{W}$ is the viewing transform. Gaussians are depth-sorted per tile and alpha-composited. Training uses a weighted combination of ℓ_1 photometric loss and D-SSIM.

2.4. Conclusion

3DGS demonstrated that explicit, rasterizable Gaussian primitives can match or exceed the image quality of leading NeRF methods while rendering at 30+ FPS in real time. It shifted the research community toward explicit representations and opened a large space of follow-on work in editing,

segmentation, compression, and generalization.

3. Approach 2: Mip-Splatting

3.1. Motivation

While 3DGS achieves high quality at training scale, it produces significant shimmering and flickering artifacts when rendered at resolutions or distances different from those seen during training. The root cause is a fundamental frequency-domain mismatch: vanilla 3DGS treats each pixel as an infinitesimal point sample, violating the Nyquist criterion when the camera zooms out or the image is downsampled. The same class of problem is addressed for textures by Mip-Maps and for implicit representations by Mip-NeRF [1]. The authors of Mip-Splatting observed that 3DGS’s 2D screen-space dilation heuristic for handling multi-scale rendering was unprincipled and produced inconsistent quality across scales.

3.2. Novelty

Mip-Splatting introduces two principled filters derived from signal processing theory, replacing the ad-hoc 2D dilation of vanilla 3DGS. The **3D smoothing filter** low-passes each Gaussian in 3D space using a box filter whose extent is determined by the maximum sampling frequency of the training cameras, preventing the representation from encoding frequencies that cannot be faithfully resolved by the training views. The **2D Mip filter** replaces the infinitesimal pixel assumption: each pixel is modeled as a finite-area sensor region, and the Gaussian’s 2D projection is convolved with this pixel footprint. Crucially, the 3D smoothing filter eliminates the problematic 2D dilation of thin Gaussians that caused needle-like artifacts at test-time scales.

3.3. Approach

For the 3D smoothing filter, each Gaussian’s covariance is convolved in 3D with $\Sigma_{\text{smooth}} = (2\pi f_{\text{max}})^{-2}\mathbf{I}$, where f_{max} is derived from the maximum camera sampling frequency across all training views. This prevents any Gaussian from being smaller than the resolution the training cameras can resolve. For the 2D Mip filter, the projected 2D covariance is augmented with the pixel footprint covariance $\Sigma_{\text{pixel}} = \delta^2\mathbf{I}$ (where δ is the pixel size in image space), yielding $\Sigma'' = \Sigma' + \Sigma_{\text{pixel}}$. As the camera zooms out and pixels subtend larger solid angles, the effective projected Gaussian grows and aliasing is naturally suppressed. The training and rasterization pipelines are otherwise identical to 3DGS; the modifications are localized to covariance computation.

3.4. Conclusion

Mip-Splatting demonstrates that the aliasing problem in 3DGS can be resolved through a principled signal-

processing treatment without sacrificing the speed or explicit nature of the representation. On MipNeRF360 and Tanks-and-Temples, Mip-Splatting outperforms vanilla 3DGS on PSNR, SSIM, and LPIPS, with the advantage growing as the downsampling factor increases. This work establishes a theoretically grounded baseline for scale-consistent Gaussian splatting.

4. Approach 3: Gaussian Grouping

4.1. Motivation

3DGS provides a significantly faster way to render 3D scenes compared to previous approaches. However, it lacks the fundamental ability to edit scenes, limiting its utility for broader applications. Every Gaussian carries information crucial for 3D reconstruction (color, opacity, and 3D coordinates), yet this information cannot determine whether two Gaussians represent the same object. Since 3DGS lacks relational information by design, Gaussian Grouping introduces a simple mechanism to add relationships between Gaussians and a training pipeline to enable scene editing. In this section we explain the authors’ approach and present results on two editing tasks: changing the color of an object and removing an object from a scene.

4.2. Novelty

The key novelty is augmenting each 3D Gaussian with a compact *identity vector*, a 16-dimensional learned embedding encoding which object instance the Gaussian belongs to. This simple addition is sufficient to enable fully instance-aware 3D segmentation, unlocking a suite of editing operations without retraining the base representation. Supervision is derived automatically from 2D segmentation masks produced by the Segment Anything Model (SAM) [4], propagated across frames by Decoupled Video Segmentation (DEVA) [2] to establish consistent cross-view identities.

4.3. Approach

Identity vectors. Each Gaussian is assigned a 16-dimensional identity vector. Gaussians representing the same object should have similar identity vectors; those representing different objects should have distinct ones. The authors report that dimension 16 is sufficient for typical scenes, though the size can be adapted for scenes with many instances.

Training pipeline. The pipeline extends standard 3DGS optimization to jointly learn scene appearance and instance identity. SAM segments each input frame, and DEVA tracks and associates objects across frames so that the same physical object receives a consistent identity label from different viewpoints. The identity feature for each pixel is rendered

differentiably via the splatting formula:

$$E_{id} = \sum_i \mathbf{e}_i \cdot \alpha_i \prod_{j < i} (1 - \alpha_j), \quad (1)$$

where $\mathbf{e}_i$ is the identity encoding of Gaussian i , α_i is its influence (derived from 2D covariance and learned opacity), and the transmittance term accounts for occlusion by depth-ordered Gaussians.

Classification layer. A linear layer projects the 16-dimensional rendered identity feature to a 256-dimensional vector (one dimension per potential object class). Softmax produces a probability distribution over instances, and cross-entropy loss against the SAM/DEVA ground-truth labels provides the training signal.

Object identification for editing. To edit a specific object, all Gaussians in the trained point cloud are classified to assign each a predicted instance ID. Visualizing distinct ID groups reveals which Gaussians correspond to the target object.

Editing operations. Color changes require only modifying the color attributes of Gaussians associated with the target ID, with no retraining needed. Object removal deletes those Gaussians from the point cloud. This leaves a hole in the scene because the occluded region was never observed in training. LaMa inpainting [7] is applied to rendered views of the edited point cloud; the inpainted frames are then used to reconstruct new Gaussians in the missing region.

4.4. Results on Our Own Data

We ran Gaussian Grouping on a self-captured desk scene containing a MacBook laptop, a water bottle, and a pair of earphones, captured with a phone camera.

Object recoloring. Figure 6 shows the rendered scene after recoloring the laptop to red. The task succeeded: the model correctly distinguished the laptop body from the Apple logo, which remained unchanged, demonstrating fine-grained instance understanding.

Object removal. Figure 7 shows the point cloud after removing the laptop Gaussians. A hole is visible where the laptop was, and the Apple logo remains because SAM identified it as a separate object. The lateral edges of the laptop were not fully removed because most capture viewpoints were taken from above, leaving the laptop’s profile under-observed. Figure 8 shows the LaMa inpainting result: despite the imperfect Gaussian removal, LaMa plausibly fills in the desk surface.

4.5. Failure Case

Figure 9 illustrates a failure case arising from imperfect SAM segmentation. At certain viewing angles, SAM mistook the shadow cast by the water bottle for a distinct object overlapping the laptop. Gaussians in that region were assigned a different identity, and the recoloring task left a

patch of the original laptop color visible from those viewpoints.

4.6. Discussion

Gaussian Grouping introduces a simple but effective mechanism for instance-level scene understanding and editing. However, three main drawbacks require attention. First, the k -nearest-neighbor regularization on identity vectors is sensitive to hyperparameter choices: a poorly chosen k , learning rate, or number of training epochs can trigger a flood-fill effect where the entire scene collapses to a single identity group. Second, as a consequence of this instability, Gaussian Grouping cannot be trained for as many epochs as vanilla 3DGS, limiting reconstruction quality relative to the baseline. Third, the method is fundamentally bottlenecked by SAM: segmentation errors propagate directly into the identity training signal, and identifying the correct instance in complex scenes requires iterating through many visualized ID groups.

5. Comparison

5.1. Commonalities and Differences

All three methods share the same foundational representation: 3D Gaussian primitives optimized via differentiable tile-based rasterization from posed 2D images. Mip-Splatting and Gaussian Grouping are both direct extensions of vanilla 3DGS. Mip-Splatting modifies covariance computation during rasterization while keeping the rest of the pipeline intact. Gaussian Grouping adds a parallel rendering stream for identity features alongside color rendering, supervised by externally generated segmentation masks. Neither extension requires changes to the densification strategy or the alpha-compositing renderer.

The primary difference lies in what each paper treats as 3DGS’s principal limitation. Mip-Splatting diagnoses a signal-processing deficiency and resolves it through principled frequency-domain filters, a passive improvement that makes rendering more faithful without changing what the scene *can express*. Gaussian Grouping diagnoses a semantic deficiency and resolves it through a learned embedding and 2D supervision, an active extension that changes what downstream tasks become possible.

5.2. Strengths

3DGS offers real-time rendering (>30 FPS), competitive image quality, and an explicit representation that is interpretable and directly manipulable. **Mip-Splatting** eliminates scale-dependent aliasing with minimal computational overhead and is theoretically principled, yielding consistent quality across rendering resolutions. **Gaussian Grouping** enables a rich family of editing operations (recoloring, removal, style transfer) without retraining, and leverages

powerful off-the-shelf models (SAM, DEVA, LaMa) in a modular pipeline.

5.3. Weaknesses

3DGS produces significant aliasing at scale changes, has no semantic understanding, and requires substantial GPU memory for complex scenes. **Mip-Splatting** does not improve semantic understanding or editing capability, and its 3D smoothing filter introduces a dependency on knowing the training camera sampling frequency, which can be difficult to estimate precisely for in-the-wild data. **Gaussian Grouping** suffers from training instability due to k -NN identity regularization, cannot match vanilla 3DGS reconstruction quality due to limited training epochs, is bottlenecked by SAM accuracy, and requires cumbersome object discovery in scenes with many instances.

5.4. Future Directions

A natural next step is combining Mip-Splatting’s anti-aliasing filters with Gaussian Grouping’s identity augmentation, yielding a representation that is simultaneously scale-consistent and semantically editable. On the identity learning side, replacing k -NN regularization with a contrastive or spatial consistency loss could improve training stability and allow longer training schedules. The SAM bottleneck could be mitigated by using 3D spatial consistency to iteratively refine imperfect 2D masks. Integrating language-grounded models for automatic object identification would make the editing workflow significantly more user-friendly.

6. Qualitative Results

6.1. MipNeRF360 Benchmark: 3DGS vs. Mip-Splatting

Figure 1 shows three viewpoints of the Garden scene at $\frac{1}{2} \times$ resolution: both methods produce visually comparable results at training scale with no perceptible difference. Figure 2 shows zoom crops at $\frac{1}{4} \times$ scale across three regions (foliage, center table, edges), along with a difference heatmap. At this scale, 3DGS exhibits visible aliasing on thin structures and flat surfaces, while Mip-Splatting produces a cleaner result. The difference heatmap confirms that errors are concentrated along edges and high-frequency regions, precisely where the absence of proper anti-aliasing is most damaging.

6.2. Own Scene: 3DGS vs. Mip-Splatting

Figure 3 and Figure 4 present results from our self-captured desk scene at $\frac{1}{8} \times$ scale. We captured the scene using a phone camera, reconstructed camera poses with COLMAP [6], and trained all three models on Google Colab Pro with a T4 GPU. The full-frame comparison confirms

3DGS vs Mip-Splatting | Garden | 7000 iters
Left: 3DGS Baseline | Right: Mip-Splatting



Figure 1. 3DGS (left) vs. Mip-Splatting (right) on the Garden scene at $\frac{1}{2} \times$ resolution across three viewpoints. Both methods produce comparable quality at training scale. We obtained the results by running the authors’ code on images from the MipNeRF360 dataset.

3DGS vs Mip-Splatting | Zoom Crop | Scale 1/4
Col 1-2: 3DGS (full+crop) | Col 3: Mip crop | Col 4: Difference heatmap

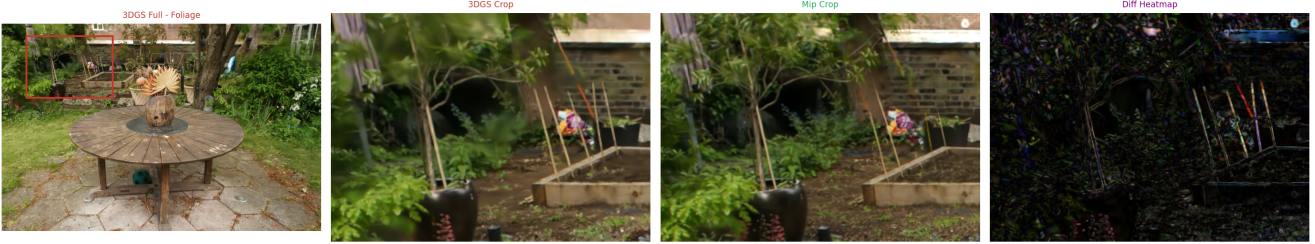


Figure 2. Zoom crop comparison at $\frac{1}{4} \times$ on the Garden scene. Rows: foliage (top), center table (middle), edges (bottom). Columns: 3DGS full frame with crop box, 3DGS crop, Mip-Splatting crop, difference heatmap. Aliasing in 3DGS is clearly visible on foliage and edges; Mip-Splatting suppresses it via its 3D smoothing and 2D Mip filters. We obtained the results by running the authors’ code on images from the MipNeRF360 dataset.

Own Scene — Frame 97 | Scale 1/8 | 10000 iters
Left: 3DGS Baseline | Center: Mip-Splatting | Right: Difference Heatmap



Figure 3. Full frame at $\frac{1}{8} \times$: 3DGS (left), Mip-Splatting (center), difference heatmap (right). Errors concentrate along object edges and the earphone wire. We obtained the results by running the authors’ code on images from our own phone camera.

Own Scene — Frame 97 | Scale 1/8 | Zoom Crop
MacBook surface — aliasing comparison



Figure 4. Zoom crop on the MacBook surface at $\frac{1}{8} \times$: 3DGS full frame, 3DGS crop, Mip-Splatting crop, difference heatmap. Mip-Splatting produces cleaner results on the flat reflective surface. We obtained the results by running the authors’ code on images from our own phone camera.

that errors concentrate along object edges and the earphone wire. The MacBook surface zoom crop confirms that Mip-Splatting produces sharper, less aliased edges on flat reflective surfaces at this extreme downsampling scale.

6.3. Own Scene: Gaussian Grouping

Figure 5 shows the identified laptop Gaussians (red) across three viewpoints of the 11228-Gaussian point cloud. Figures 6–9 show the editing results: successful recoloring and removal with LaMa inpainting, as well as the failure case caused by SAM misidentifying the water bottle shadow.

6.4. Analysis

Across both the benchmark dataset and our own scene, Mip-Splatting consistently outperforms vanilla 3DGS at downsampled rendering scales, with the most visible improvement on planar surfaces and thin structures. The difference heatmaps confirm that errors in 3DGS are concentrated along edges and high-frequency regions. On our own data, Gaussian Grouping successfully segments the main objects in the scene, although fine structures like earphone cables are not always cleanly separated. The editing tasks succeed on coarse objects like the laptop but struggle where capture coverage is incomplete, confirming that capture quality directly affects editing fidelity.

7. Conclusion

This project provided a hands-on exploration of the 3D Gaussian Splatting ecosystem through three interconnected

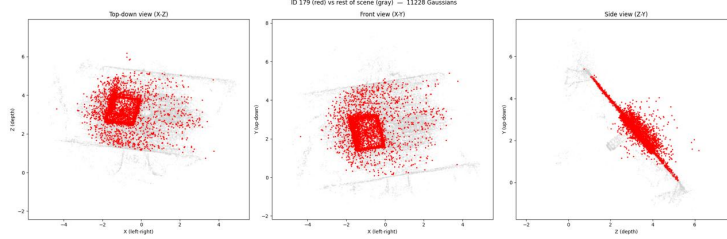


Figure 5. Laptop Gaussians (red) vs. rest of scene (gray) across top-down, front, and side views of the 11228-Gaussian point cloud. We obtained the results by running the authors’ code on images from our own phone camera.

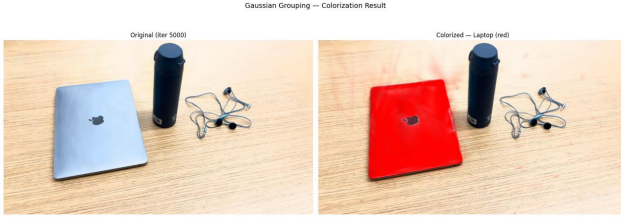


Figure 6. Original scene (left) and laptop recolored red (right). The Apple logo is correctly preserved as a separate instance. We obtained the results by running the authors’ code on images from our own phone camera.

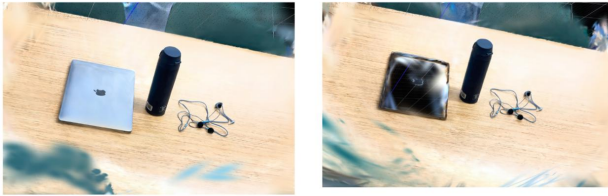


Figure 7. Original scene (left) and laptop removed (right). A hole is visible where the laptop was; the Apple logo remains as SAM identified it separately. We obtained the results by running the authors’ code on images from our own phone camera.



Figure 8. Laptop removal render (left) and LaMa inpainting (right). LaMa plausibly fills the desk surface despite imperfect edge removal. We obtained the results by running the authors’ code on images from our own phone camera.

papers. 3DGS established that explicit Gaussian primitives rendered via differentiable tile-based rasterization can achieve real-time NVS at high quality. Mip-Splatting demonstrated that a principled signal-processing treatment of rendering scale eliminates aliasing artifacts with minimal overhead. Gaussian Grouping showed that augmenting the Gaussian representation with learned identity embeddings enables instance-level scene understanding and a practical



Figure 9. Failure case: the water bottle shadow was misidentified by SAM as a separate object overlapping the laptop, leaving a patch unrecolored. We obtained the results by running the authors’ code on images from our own phone camera.

suite of editing operations.

Together, these methods illustrate the maturity and versatility of the 3DGS paradigm, as well as the remaining open problems: memory cost for complex scenes, training instability from auxiliary objectives, sensitivity to capture coverage, and the need for semantically guided editing interfaces. Future work combining Mip-Splatting’s anti-aliasing with more stable identity learning and language-grounded editing interfaces is a promising direction.

8. Summary of Contributions

Dhwani Auvnish Chande led the Introduction and Mip-Splatting sections, implemented the full training and evaluation pipeline for Papers 1 and 2 on Google Colab, conducted multi-scale rendering experiments and metric computation (PSNR, SSIM, LPIPS), and captured and processed the own-data scene using a phone camera and COLMAP.

Trong Gia Hung Nguyen wrote the Gaussian Grouping section, implemented the full Gaussian Grouping pipeline for Paper 3 including identity vector training, object recoloring, object removal, and LaMa inpainting, and produced all qualitative results for the editing experiments.

Nishita Konduru was responsible for report editing and formatting.

References

- [1] Jonathan T. Barron, Ben Mildenhall, Dor Verbin, Pratul P. Srinivasan, and Peter Hedman. Mip-NeRF 360: Unbounded anti-aliased neural radiance fields. In *Proceedings*

of the *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

- [2] Ho Kei Cheng, Seoung Wug Oh, Brian Price, Alexander Schwing, and Joon-Young Lee. Tracking everything everywhere all at once. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [3] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D Gaussian splatting for real-time radiance field rendering. In *ACM SIGGRAPH 2023 Conference Proceedings*, 2023.
- [4] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C. Berg, Wan-Yen Lo, Piotr Dollár, and Ross Girshick. Segment anything. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [5] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2020.
- [6] Johannes L. Schönberger and Jan-Michael Frahm. Structure-from-motion revisited. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [7] Roman Suvorov, Elizaveta Logacheva, Anton Mashikhin, Anastasia Remizova, Arsenii Ashukha, Aleksei Silvestrov, Naejin Kong, Harshith Goka, Kiyon Park, and Victor Lempitsky. Resolution-robust large mask inpainting with Fourier convolutions. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2022.
- [8] Mingqiao Ye, Martin Danelljan, Fisher Yu, and Lei Ke. Gaussian grouping: Segment and edit anything in 3D scenes. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2024.
- [9] Zehao Yu, Anpei Chen, Binbin Huang, Torsten Sattler, and Andreas Geiger. Mip-splatting: Alias-free 3D Gaussian splatting. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.